



Google Summer of Code 2021

[Cilium]

Project: Expose Cilium Config to CLI Tool

Personal Info

Name: Gaurav Genani

Email: h3llix.pvt@gmail.com

Telephone: +91 8717874888

Time Zone: India (UTC+5.30)

GitHub: [h3llix](#)

Slack username: Gaurav Genani

University Info

University: Indian Institute of Technology, Roorkee

Major: Chemical Engineering

Current Year: 2nd year (2023 expected graduation)

Degree: Bachelor of Technology (B.Tech)

About Me

I am a 2nd Year **Bachelor of Technology** student at the **Indian Institute of Technology, Roorkee**. I've been involved in software development for the past 1.5 years, with several projects deployed to production. I am a member of our University's software development group (SDSLabs) and have been involved in software development in various spheres, including cloud and infra development and information security.

I've been passionate about programming and software development since my school days and have worked on several personal and group projects since coming to college. In my freshman year, I was introduced to Docker and Kubernetes and programming in Go in general. I am also involved with information security and am an active CTF player with InfoSecIITR, my university's CTF team.

Statement of Motivation

What is your motivation for participating in Google Summer of Code?

I have always been excited by the prospect of converting ideas into products with real-world impact and was captivated by the concept of open source ever since I was first introduced to it. Since then, I have wanted to be a part of this community and have been trying to contribute in the small ways I could by having all my projects on GitHub to welcome contributions in my projects. GSoC provides the perfect opportunity to jump right into the Open Source World with the opportunity to work on awesome and even bigger open-source projects with experienced developers. That's why I am motivated to participate in GSoC, as it will enhance my knowledge and development skills and give me an opportunity to work on industry-scale projects under experienced mentors.

Why did you choose Cilium, and why this project idea?

I am a member of SDSLabs, a student technical club at IIT Roorkee that constantly tries to innovate and foster software development activities on the campus. One of my



seniors in SDS Labs has been a core contributor to Cilium (Deepesh), and he introduced me to eBPF. After reading about eBPF, I was amazed by its possibilities, and that excites me. Also, I was amazed to know about the supportive community of Cilium by hearing [fristonio's](#) (Deepesh) experience and eventually realizing it myself as I worked with my mentors.

I chose this particular project because I am more comfortable in golang, which perfectly aligns with my interests. Also, I wanted to explore more on eBPF, and this project provides an excellent opportunity to apply my knowledge and skills on a practical scale. I always wanted to optimize the throughput of software by analyzing its performance bottlenecks. Extending the CLI tool in such a large-scale project helps me take a significant step towards it. This project lies in my sphere of interest, and therefore I have a natural inclination towards it.

What are your expectations from us during and after the successful completion of the program?

I am looking forward to bonding with the community and making some valuable contributions to the project. I would be grateful to join the family of Cilium. I am always open to essential discussions over ideas and their implementations. I would love to receive feedback and suggestions on my contributions, as I believe it will help improve my code quality and help me become an overall better developer in the long run.

As I mentioned above, your community's projects as a whole drive me to make consistent contributions to the work it is doing. The GSoC project allows me to work on the project as a whole, and after its successful completion, I would be keen to take up more important issues and continue my open-source contribution through Cilium.

What are you hoping to learn?

I expect to build upon my DevOps skillset by setting up a large project. Cilium could help me more with learning eBPF and core networking concepts. I hope to gain a deeper understanding of the datapath which a packet follows and how a bpf program hooks itself on the kernel level. My work on `cilium CLI` would allow me to understand how `cilium daemon` works in general and the configurations that daemon currently supports.

Also, staying in continuous touch with such qualified people in the community would be an excellent opportunity for me. I am very excited to work on this project, along with the highly supportive Cilium community in the coming summers.

Related Experience

What kind of projects have you worked on in the past? What technologies did you use?

- [Gasper](#) is an intelligent Platform as a Service (PaaS) written in golang used for deploying and managing applications and databases inside Docker containers.
- [Pinger](#) is an open-source implementation of an application that can create status pages. Some similar applications are Freshping and Apex Ping. This project is currently under development.
- I have interned at [Happiest Neurons](#), where I was responsible for managing the infrastructure and handling deployments in addition to backend development. My work revolved around integrating user authentication using AWS Cognito and API gateway.
- [CricLive](#) was a team project built on Unity as a submission for InOut 7.0. It is a revolutionary application that shows live cricket matches by using text commentary and then using Artificial Intelligence renders the live match using animations(video). We were also the Runners up in InOut 7.0 this year. [Video](#)
- Slack-Bot is a fun bot that usually helps us with little fun things in our workspace. It is entirely written in Golang.
- Being part of several projects under SDS Labs, I feel comfortable with Golang, C++, Python, C#. I have sound knowledge of Docker and Kubernetes. Apart from development, I am an active CTF player as well.

Information Security Related :

1. **CSAW ESC** is one of the oldest hardware security competitions in the world where we were supposed to reverse the kernel of IoT devices based on RISC-V. We were 1st in India and the second runner-up in the world.
2. **InterIIT Tech Meet Gold** - I participated in InterIIT Tech Meet in Saptang-Labs security hackathon and made reports of some non-disclosable CVEs with my team of 5. I made a report and wrote some tests for explaining the vulnerability of apache tomcat servers in a Windows VM.

Open Source Contributions:

- Gasper: I worked on gasper's service, which deploys and manages databases and manages their life cycle. Also added support for deploying [Redis](#) and [PostgreSQL](#) databases among the nodes.

<https://github.com/sdslabs/gasper/pull/202>

<https://github.com/sdslabs/gasper/pull/198>

- Pinger: I added [UDP](#) and [WebSocket](#) checkers, which validate the probe and checks the response received. Also, I wrote the [exporter](#) for exporting metrics to the Influx DB.

<https://github.com/sdslabs/pinger/pull/39>

<https://github.com/sdslabs/pinger/pull/38>

<https://github.com/sdslabs/pinger/pull/83>

Contributions on Cilium

I have worked on the bug tool in two of my PR's, some documentation changes and logging errors.

- <https://github.com/cilium/cilium/pull/15363> - (merged)
- <https://github.com/cilium/cilium/pull/15525> - (merged)
- <https://github.com/cilium/cilium/pull/15550> - (opened)
- <https://github.com/cilium/cilium/pull/15380> - (merged)
- <https://github.com/cilium/cilium/pull/15339> - (merged) (This pr was reverted due to failing tests [#15525](#) is the new pr)

Project Details

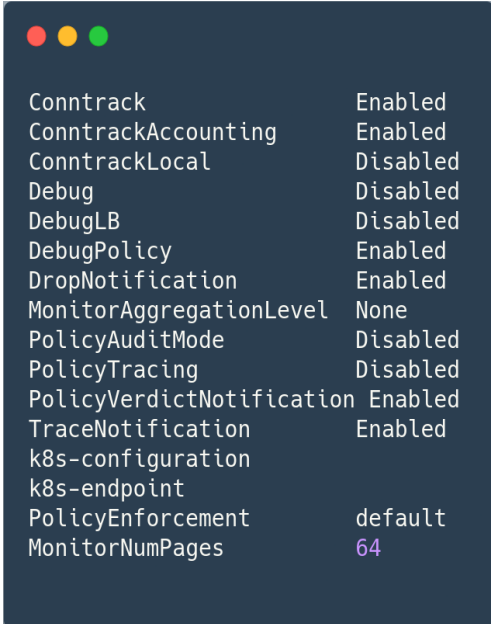
Overview

This project aims at improving the cilium-agent cli, which is exposing the active cilium-agent configurations through the cli. The ***cilium config*** command will be modified to provide all the active configurations.

Extending *cilium config*

Currently, the cilium-agent cli only exposes a subset of daemon configurations. This project aims to expose all the active daemon configurations through the Cilium config. Currently, there are three ways by which we can see the active or operational configurations of cilium-agent :

1. **By using the command ``cilium config``** - When we try to run ***cilium config***, it shows the below configurations as output.

A terminal window with a dark blue background and three colored window control buttons (red, yellow, green) in the top left corner. The terminal displays the output of the 'cilium config' command, showing various configuration options and their status.

```
Conntrack                Enabled
ConntrackAccounting      Enabled
ConntrackLocal           Disabled
Debug                    Disabled
DebugLB                  Disabled
DebugPolicy              Enabled
DropNotification         Enabled
MonitorAggregationLevel  None
PolicyAuditMode          Disabled
PolicyTracing            Disabled
PolicyVerdictNotification Enabled
TraceNotification        Enabled
k8s-configuration
k8s-endpoint
PolicyEnforcement        default
MonitorNumPages          64
```

Most of the above are datapath-specific configurations and do not give an idea about all the active daemon configurations.

2. **Using cilium-agent logs** - Most of the active configurations can be seen at the start of `cilium-agent's` log, which is the configurations at the start of the daemon, But these logs can be rotated out, furthermore, if there is a change in the daemon configuration, these logs cannot tell us about the updated configurations.

```

2021-04-09T12:39:35.407644687Z level=info msg=" --agent-health-port='9876'" subsys=daemon
2021-04-09T12:39:35.407652344Z level=info msg=" --agent-labels=''" subsys=daemon
2021-04-09T12:39:35.407654979Z level=info msg=" --allow-icmp-frag-needed='true'" subsys=daemon
2021-04-09T12:39:35.407657110Z level=info msg=" --allow-localhost='auto'" subsys=daemon
2021-04-09T12:39:35.407659177Z level=info msg=" --annotate-k8s-node='true'" subsys=daemon
2021-04-09T12:39:35.407661187Z level=info msg=" --auto-create-cilium-node-resource='true'"
subsys=daemon
2021-04-09T12:39:35.407663202Z level=info msg=" --auto-direct-node-routes='false'" subsys=daemon
2021-04-09T12:39:35.407665174Z level=info msg=" --blacklist-conflicting-routes='true'" subsys=daemon
2021-04-09T12:39:35.407667171Z level=info msg=" --bpf-compile-debug='false'" subsys=daemon
2021-04-09T12:39:35.407669095Z level=info msg=" --bpf-ct-global-any-max='262144'" subsys=daemon
2021-04-09T12:39:35.407671016Z level=info msg=" --bpf-ct-global-tcp-max='524288'" subsys=daemon
2021-04-09T12:39:35.407672944Z level=info msg=" --bpf-ct-timeout-regular-any='1m0s'" subsys=daemon
2021-04-09T12:39:35.407674875Z level=info msg=" --bpf-ct-timeout-regular-tcp='6h0m0s'" subsys=daemon
2021-04-09T12:39:35.407676821Z level=info msg=" --bpf-ct-timeout-regular-tcp-fin='10s'" subsys=daemon
2021-04-09T12:39:35.407678767Z level=info msg=" --bpf-ct-timeout-regular-tcp-syn='1m0s'" subsys=daemon
2021-04-09T12:39:35.407680711Z level=info msg=" --bpf-ct-timeout-service-any='1m0s'" subsys=daemon
2021-04-09T12:39:35.407682626Z level=info msg=" --bpf-ct-timeout-service-tcp='6h0m0s'" subsys=daemon
2021-04-09T12:39:35.407684581Z level=info msg=" --bpf-fragments-map-max='8192'" subsys=daemon
2021-04-09T12:39:35.407707056Z level=info msg=" --bpf-map-dynamic-size-ratio='0.0025'" subsys=daemon
2021-04-09T12:39:35.407714320Z level=info msg=" --bpf-nat-global-max='524288'" subsys=daemon
2021-04-09T12:39:35.407716945Z level=info msg=" --bpf-ne

```

3. **Viewing the active configmap in Kubernetes:**

View :

Most of the applied configurations can be seen by viewing the configmap in the Kubernetes cluster. This can be done by typing **kubectl -n kube-system get configmap cilium-config -o yaml**, which is shown as below.

```
kubectl -n kube-system get configmap cilium-config -o yaml
apiVersion: v1
data:
  auto-direct-node-routes: "false"
  bpf-lb-map-max: "65536"
  bpf-map-dynamic-size-ratio: "0.0025"
  bpf-policy-map-max: "16384"
  cilium-endpoint-gc-interval: 5m0s
  cluster-id: ""
  cluster-name: default
  cluster-pool-ipv4-cidr: 10.0.0.0/8
  cluster-pool-ipv4-mask-size: "24"
  custom-cni-conf: "false"
  debug: "false"
  disable-cnp-status-updates: "true"
  enable-auto-protect-node-port-range: "true"
  enable-bandwidth-manager: "false"
  enable-bpf-clock-probe: "true"
  enable-bpf-masquerade: "true"
  enable-endpoint-health-checking: "true"
  enable-health-check-nodeport: "true"
  enable-health-checking: "true"
```

Issue:

The main issue is that ConfigMap does not stay reconciled with the active daemon configurations. This means that if we change any configuration in the config map, it is not immediately reflected in Cilium. Cilium needs to be restarted to apply those configurations. So it may happen that ConfigMap and daemon both show different active configurations.

Solution:

View the active configurations in the cilium-agent. This project aims at exposing all the active configurations via cilium-agent cli.

Also, it may so happen that the ConfigMap values and active cilium-agent configurations might differ depending on the eBPF features supported by the underlying kernel. This could be understood with the help of the code snippet below.


```

// strict denotes to panic if any to-be enabled feature cannot be enabled
strict = option.Config.KubeProxyReplacement != option.KubeProxyReplacementProbe

if option.Config.KubeProxyReplacement == option.KubeProxyReplacementProbe ||
   option.Config.KubeProxyReplacement == option.KubeProxyReplacementStrict {

    log.Infof("Trying to auto-enable %q, %q, %q, %q, %q features",
        option.EnableNodePort, option.EnableExternalIPs,
        option.EnableHostReachableServices, option.EnableHostPort,
        option.EnableSessionAffinity)

    option.Config.EnableHostPort = true
    option.Config.EnableNodePort = true
    option.Config.EnableExternalIPs = true
    option.Config.EnableHostReachableServices = true
    option.Config.EnableHostServicesTCP = true
    option.Config.EnableHostServicesUDP = true
    option.Config.EnableSessionAffinity = true
}

```

Depending on the mode of KubeProxyReplacement, certain configurations are changed or applied by default. This could create some inconsistency between the applied configurations and the active configurations.

Thus extending `cilium config` would help with debugging live environments based on the configuration settings that are currently active.

`cilium encrypt status` and `cilium encrypt flush.`

This project will also add a new subcommand, `cilium encrypt`, to the CLI with two subcommands - `status` and `flush`.

Where `cilium encrypt status` should display information on the IPsec configuration's current status, and `cilium encrypt flush` would flush all XFRM states (`ip xfrm state flush` and `ip xfrm policy flush`). This command is already needed by the community and has been marked as issue [#14638](#).

Implementation Details

Extending `cilium config` (Need discussion todo)

1. **DaemonConfig** struct contains all the cilium-agent related config.

```
type DaemonConfig struct {
    BpfDir      string    // BPF template files directory
    LibDir      string    // Cilium library files directory
    RunDir      string    // Cilium runtime directory
    NAT46Prefix *net.IPNet // NAT46 IPv6 Prefix
    Devices     []string  // bpf_host device
    DirectRoutingDevice string    // Direct routing device (used only by NodePort BPF)
    .....
    .....
    // EnableCustomCalls enables tail call hooks for user-defined custom
    // eBPF programs, typically used to collect custom per-endpoint
    // metrics.
    EnableCustomCalls bool
}
```

2. Firstly, we extend the **DaemonConfigurationStatus** struct by changing the OpenAPI specification to include more configurations, and Since there is already an endpoint to get the configurations, there is no need for a new endpoint. So we extend the current definitions, which would allow more daemon configuration fields to be exposed.
3. For the sake of explanation, I am creating a field called **daemonConfig**. We could use free form objects in open API to generate a field in **DaemonConfigurationStatus** as shown below :

```
DaemonConfigurationStatus:
  description: |
    ....
    daemonConfig:
      <Properties need to be added>
```

4. Now we need to generate the API server, client, and updated models using **make generate-api**. This would create a field called **DaemonConfig** field in **DaemonConfigurationStatus**.

```

type DaemonConfigurationStatus struct {
    ....
    DaemonConfig map[string]interface{} `json:"daemonConfig,omitempty"`
    ....
}

```

5. We can also change the type to be something like `map[string]somejsontype`. We can use the JSON type at <https://pkg.go.dev/k8s.io/apiextensions-apiserver/pkg/apis/apiextensions/v1#JSON>.
6. Instead of adding all the configurations to **DaemonConfigurationStatus**, we defined a map that would contain all the cilium-agent configurations. The **DaemonConfig** struct, which contains all the cilium-agent configurations, needs to be passed as a map to the above newly created field. This can be done by annotating the fields in **DaemonConfig** with JSON and sending them as payload to API.
7. The middleware Responder for the endpoint, which gets the **DaemonConfigurationStatus**, needs to be adjusted accordingly.
8. Post the previous step; we modify the **cilium/cmd/config.go**. The response here now contains a map that has all the active daemon configurations. If required, a parser will be written at this end.

```

func configDaemon(cmd *cobra.Command, opts []string) {
    dOpts := make(models.ConfigurationMap, len(opts))

    resp, err := client.ConfigGet()
    if err != nil {
        Fataalf("Error while retrieving configuration: %s", err)
    }
    ....

    dumpConfig(cfgStatus.Immutable)
    dumpConfig(cfgStatus.Realized.Options)
    fmt.Printf("%-24s %s\n", "bpf-dir", cfgStatus.BpfDir)
    fmt.Printf("%-24s %s\n", "k8s-configuration", cfgStatus.K8sConfiguration)
    ....
}

```

Discussion needed on implementation and how to expose configurations as read-only:

Proposed way :

On digging a little deeper into the codebase of cilium-agent cli codebase, following relevant code sections, which can be used to make the configurations read-only. The Immutable field in the ConfigurationStatus looks promising:

```
type DaemonConfigurationStatus struct {
    // addressing
    Addressing *NodeAddressing `json:"addressing,omitempty"`
    ...
    // Immutable configuration (read-only)
    Immutable ConfigurationMap `json:"immutable,omitempty"`
    ...
    // MTU for network facing routes
    RouteMTU int64 `json:"routeMTU,omitempty"`
}
```

The DaemonConfigurationStruct contains an Immutable field that holds the read-only configurations.

```
// Option is the structure used to specify the semantics of a configurable
// boolean option
type Option struct {
    // Define is the name of the #define used for BPF programs
    Define string

    // Immutable marks an option which is read-only
    Immutable bool
    ...
}
```

Thus a configurable option can be defined by setting the immutable bool as true.

```

package option

var (
    ...

    specBpfDir = Option{
        Description: "gsoc test",
        Immutable:   true,
    }

    // DaemonOptionLibrary is the daemon's option library that should be
    // used for read-only.
    DaemonOptionLibrary = OptionLibrary{
        PolicyTracing: &specPolicyTracing,
    }

    DaemonMutableOptionLibrary = OptionLibrary{
        ...
    }

    DaemonImmutableOptionLibrary = OptionLibrary{
        "BpfDir": &specBpfDir,
    }
)

func init() {
    for k, v := range DaemonMutableOptionLibrary {
        DaemonOptionLibrary[k] = v
    }

    for k, v := range DaemonImmutableOptionLibrary {
        DaemonOptionLibrary[k] = v
    }
}

// ParseDaemonOption parses a string as daemon option
func ParseDaemonOption(opt string) (string, OptionSetting, error) {
    return ParseOption(opt, &DaemonOptionLibrary)
}

```

For the sake of explanation, I created an option for **BpfDir** and set **immutable** bool as **true**. Above are suggested changes; the **DaemonImmutableOptionLibrary** is mapped to **DaemonOptionLibrary** and is passed to the **NewIntOptions** function.

```

var (
    // Config represents the daemon configuration
    Config = &DaemonConfig{
        Opts:                                NewIntOptions(&DaemonOptionLibrary),

```

Now I can find a function called ***GetMutableModel*** exists. But ***GetImmutableModel*** function returns an empty configuration map. So this needs to be implemented, which filters options based on ***immutable*** bool.

```

// GetImmutableModel returns the set of immutable options as a ConfigurationMap API model.
func (o *IntOptions) GetImmutableModel() *models.ConfigurationMap {
    immutableCfg := make(models.ConfigurationMap)

    // basically we need to implement this where a configuration map is returned corresponding
    // corresponding to those configuration whose immutable boolean is false
    return &immutableCfg
}

// GetMutableModel returns the set of mutable options as a ConfigurationMap API model.
func (o *IntOptions) GetMutableModel() *models.ConfigurationMap {
    mutableCfg := make(models.ConfigurationMap)
    o.optsMU.RLock()
    for k, v := range o.Opts {
        _, config := o.Library.Lookup(k)
        ....
    }
}

```

Finally, the API Handler is modified as shown below to use the ***GetImmutableModel*** function.

```
func (h *getConfig) Handle(params GetConfigParams) middleware.Responder {
    log.WithField(logfields.Params, logfields.Repr(params)).Debug("GET /config request")

    ....

    status := &models.DaemonConfigurationStatus{
        Addressing:      node.GetNodeAddressing(),
        ....
        Immutable:       *option.Config.Opts.GetImmutableModel(),
        BpfDir:          option.Config.GetBPFDir(),
        ....
    }

    cfg := &models.DaemonConfiguration{
        Spec:    spec,
        Status:  status,
    }
}
```

Cilium encrypt

<https://github.com/vishvananda/netlink> gives the already required information. And this could be taken <https://github.com/cilium/cilium/pull/14484> as a reference.

Deliverables

1. Exposing daemon configurations:

- ***cilium config*** will show all the active configurations in the cilium agent. For the scope of this project, configurations will be listed in alphabetical order. Earlier I decided to use a flag to categorize configurations, but in view of this pr <https://github.com/cilium/cilium/pull/15564> I dropped that and have an idea to propose

Idea: We could use tags to categorize the configurations. For example, we could assign multiple tags to the configurations. Let's say a configuration was assigned bpf and datapath tags. Now the user can search for configurations using multiple tags. **NOTE:** This is beyond the scope of the Gsoc Proposal.

- Now cilium-agent cli would be able to provide the value of a specific option. For example, ***cilium config enable-host-reachable-services*** will directly give the value from cli.
- Some configurations present in `cilium status` will be moved to `cilium config`, for example, `KubeProxyReplacement`.
- Since cilium-agent cli would now provide the value of option, we could replace now the various places where we use cilium status to infer which features are enabled e.g, below

```
func (kub *Kubect1) HasBPFNodePort(pod string) bool { status :=
kub.CiliumExecContext(context.TODO(), pod,
    "cilium status -o
jsonpath='{.kube-proxy-replacement.features.nodePort.enabled}')
```

Can be replaced as:

```
func (kub *Kubect1) HasBPFNodePort(pod string) bool { status :=
kub.CiliumExecContext(context.TODO(), pod,
    "cilium config kube-proxy-replacement")
```

In case of failing tests, we can also dump the cilium configurations as it would give more insights on what caused the failure of the test. Such as here


```
AfterFailed(func() {
    kubect1.CiliumReport("cilium status", "cilium endpoint
list","cilium config --verbose")
})
```

2. Adding subcommand `cilium encrypt status` and `cilium encrypt flush.`

cilium encrypt status would now show the IPsec related information as stated below

```
cilium encrypt status
Encryption: ipsec
NodeEncryption: disabled
Keys: 1/1
Packets: 5555 rcvd / 6666 sent
Errors: None
```

cilium encrypt flush would flush all XFRM states.

Project Timeline

Pre GSoC Period	
April 14th - May 16th	- I would be completing my remaining PRs on Cilium and try to complete my assigned issues before the community bonding period. - Resolving new issues Reading more on cilium's architecture
Community Bonding Period	
May 17th - June 6th	Get to know more about the Cilium community.

	• Discussion with my mentor to discuss methods on implementation. • Identify more relevant part in the codebase
Coding Period	
June 7th - June 14th	• Getting familiar with existing cilium configurations • Create models for configurations
June 15th - June 26th	• Add models to open API • Write middlewares for sending payloads
	Buffer time to complete previous tasks(if any)
June 29th - July 9th	• Start working on separating Mutable and Immutable configurations.
July 10th - July 11th	Buffer time to complete previous tasks(if any)
July 12th - July 16th	Evaluation 1 Phase
July 17th - July 23rd	• Work on implementation of the cilium encrypt status
July 24th - August 5th	• Work on implementation of the cilium encrypt flush.
6th August - 12th August	• If needed any, CLI UX fixes. • Writing tests and documentation if required.
13th August - 15th August	Buffer time to complete previous tasks(if any)
16th Aug - 23rd Aug	Debug and Testing

Availability

I would be easily able to dedicate more than 30 hours a week. The college's summer vacations have been shifted ahead to the entire months of June, July and extending to the first week of August. Therefore, there would be no classes for the first two months of the program's coding period and no exams throughout the coding period. My college would reopen from Aug 4th. During this time also, I would easily be able to work more than 25 hours a week.

I generally work from 11 AM to 1 AM IST, although I find it flexible to adjust my working hours if required.

Other than this project, I have no commitments/vacations planned for the summer. I shall keep my status posted to all the community members every week and maintain transparency in the project.

Why me?

I've been quite enthusiastic and eager to learn new things and try my hand at new technologies. I've already worked on a couple of cloud projects like Gasper, Pinger, and slack-bot and have been involved in programming in Go for over one year; hence, I can apply the same knowledge and experience drawn from there. I am confident that I can contribute at par to the standards an open-source organization demands.

Apart from this, I would get to work with many professionals as my mentors. This will be a huge learning experience for me. Having worked in numerous teams and open source projects, I am aware of the software development process and the best practices and recommendations.

Involvement after GSoC

I have already learned a lot contributing to the Cilium. Even after the GSoC period ends, I plan on contributing to this organization by adding to my past projects and working on open issues because of the familiarity of the technical stack and the new challenges that I am continually offered in the process.

Also, I would like to complete the secondary goals mentioned in my proposal. Having picked up many development skills, my primary focus would be to help the project and the community grow.

Ideas to work on after my primary goals are completed:

Note: This is beyond the scope of my GSoC project.

1. Hot reloading of daemon configurations without restarting the agent.

Pros: In such cases, applied and active daemon configurations will remain in sync.

Cons: Sudden change in configurations might cause packet loss.

2. Currently, what I can infer is cilium-agent runs on each node locally, as mentioned [here](#). But can we extend cilium-agent cli to query on a cluster-wide basis and run commands on multiple nodes at once

A possible approach that is coming to mind is to use Kubernetes go driver to list pods by filtering out those unrelated to Cilium and running the queries on filtered pods at once.

References

- <https://docs.cilium.io/en/v1.9/concepts/>
- <https://github.com/cilium/cilium/commit/e5e7b02ab760b49cad235d5315ffcd7e698ef224>
- <https://github.com/cilium/cilium/commit/c285ca04757f6caed38c45f84cbe255ce4d34c25>